I focused on optimizing machine learning models through hyperparameter tuning and exploring Automated Machine Learning (AutoML) with AutoGluon. Before this lab, I mainly worked with default model settings, so it was helpful to clearly separate model parameters—which the algorithm learns on its own from the data—from hyperparameters, which are the structural settings we configure before training begins, like the number of trees (n_estimators) or maximum tree depth (max_depth) in a Random Forest.

Working with the Iris dataset, my baseline Random Forest classifier actually hit 100% accuracy on the test split right out of the gate. However, running GridSearchCV and RandomizedSearchCV showed why cross-validation is so important for getting a realistic measure of how a model generalizes. Both tuning methods achieved a 94.29% cross-validated accuracy across 5 folds, though they arrived there differently. Grid Search found an optimal combination of max_depth: 5 and n_estimators: 100 by testing every single option in a small grid (45 total fits). Random Search reached the same 94.29% score using n_estimators: 200, min_samples_split: 5, and max_depth: 20 by sampling just 10 combinations across a much wider parameter space (50 total fits). Seeing them side-by-side made the practical trade-off clear: Grid Search is great when you have a small, specific set of values and want a guaranteed best combination, whereas Random Search is much more practical when exploring a large parameter space under time or compute limits.

The highlight of the lab was using AutoGluon's TabularPredictor. With just a few lines of code and a 60-second time limit, it automated data preprocessing, feature generation, training across 11 diverse models—including LightGBM, CatBoost, XGBoost, Random Forests, Extra Trees, and Neural Networks—and combined them into a multi-layer weighted ensemble (WeightedEnsemble_L2). Rather than spending hours manually testing individual algorithms and tweaking settings one by one, AutoML handled the entire pipeline at once. Going forward, I can see myself using AutoML to quickly build strong baselines and identify the best-performing model families, then using targeted Random or Grid Search when I need fine-grained control over a specific model.